DATA STRUCTURES USING C

Lab 1:

main.c

```c
#include <stdio.h>

int main() {
    int a[10], n, i, max, min;

    scanf("%d", &n);

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    max = min = a[0];

    for(i = 1; i < n; i++) {
        if(a[i] > max) max = a[i];
        if(a[i] < min) min = a[i];
    }

    printf("%d %d", max, min);
    return 0;
}
```

input

```
5
7 8 1 2 3
8 1
```

LAB 2:

main.c

```c
#include <stdio.h>

int fact(int n) {
    if (n == 0)
        return 1;
    return n * fact(n - 1);
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%d", fact(n));
    return 0;
}
```

input

```
5
120

...Program finished with exit code 0
Press ENTER to exit console.
```

LAB 3:

main.c

```c
#include <stdio.h>

int fib(int n) {
    if(n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int n;
    scanf("%d", &n);
    printf("%d", fib(n));
    return 0;
}
```

input

```
10
55

...Program finished with exit code 0
Press ENTER to exit console.
```

LAB 4: Singly linked list

Insert at beginning:

main.c Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct Node{
    int data;
    struct Node *next;
};
struct Node *head=NULL;
void insertBeginning(int value){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data=value;
    newNode->next=head;
    head=newNode;
}
void display(){
    struct Node *temp=head;
    while(temp!=NULL){
        printf("%d -> ",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}
int main(){
    insertBeginning(10);
    insertBeginning(20);
    insertBeginning(30);
    display();
}
```

```
Output
30 -> 20 -> 10 -> NULL
```

Insert at End:

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node{
    int data;
    struct Node *next;
};
struct Node *head=NULL;
void insertEnd(int value){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data=value;
    newNode->next=NULL;
    if(head==NULL){
        head=newNode;
        return;
    }
    struct Node *temp=head;
    while(temp->next!=NULL)
        temp=temp->next;
    temp->next=newNode;
}
void display(){
    struct Node *temp=head;
    while(temp!=NULL){
        printf("%d -> ",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}
int main(){
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    display();
}
```

```
Output
10 -> 20 -> 30 -> NULL
```


Delete Node:

main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct Node{
    int data;
    struct Node *next;
};
struct Node *head=NULL;
void insertEnd(int value){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node
        ));
    newNode->data=value;
    newNode->next=NULL;
    if(head==NULL){
        head=newNode;
        return;
    }
    struct Node *temp=head;
    while(temp->next!=NULL)
        temp=temp->next;
    temp->next=newNode;
}
void deleteBeginning(){
    struct Node *temp=head;
    head=head->next;
    free(temp);
}
void display(){
    struct Node *temp=head;
    while(temp!=NULL){
        printf("%d -> ",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}
int main(){
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    deleteBeginning();
    display();
}
```

Output

```
20 -> 30 -> NULL
```

LAB 5:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node* top = NULL;
void push(int x) {
    struct Node* new = malloc(sizeof(struct Node));
    new->data = x;
    new->next = top;
    top = new;
}
int pop() {
    if (top == NULL) return -1;
    int val = top->data;
    struct Node* temp = top;
    top = top->next;
    free(temp);
    return val;
}
```

```c
int peek() {
    if (top == NULL) return -1;
    return top->data;
}
int main() {
    push(10);
    push(20);
    push(30);
    printf("Top: %d\n", peek());
    printf("Popped: %d\n", pop());
    printf("Top after pop: %d\n", peek());
    return 0;
}
```

```
Top: 30
Popped: 30
Top after pop: 20
```

LAB 6:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* next;
};
struct Node *front = NULL, *rear = NULL;
void enqueue(int x) {
    struct Node* new = malloc(sizeof(struct Node));
    new->data = x;
    new->next = NULL;
    if (rear == NULL) {
        front = rear = new;
        return;
    }
    rear->next = new;
    rear = new;
}
int getFront() {
    if (front == NULL) return -1;
    return front->data;
}
```

```c
int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);
    printf("Front: %d\n", getFront());
    return 0;
}
```

```
Front: 10
```

LAB 7: Doubly Linked list
Insert at Beginning:

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node *head=NULL;
void insertBeginning(int value){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=value;
    newNode->prev=NULL;
    newNode->next=head;
    if(head!=NULL)
        head->prev=newNode;
    head=newNode;
}
void display(){
    struct Node *temp=head;
    while(temp!=NULL){
        printf("%d <-> ",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}
int main(){
    insertBeginning(10);
    insertBeginning(20);
    insertBeginning(30);
```

```
Output

10 -> 30 -> NULL
```

Insert At End:

main.c

```c
#include<stdio.h>
#include<stdlib.h>
struct Node{
    int data;
    struct Node *next;
    struct Node *prev;
};
struct Node *head=NULL;
void insertEnd(int value){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=value;
    newNode->next=NULL;
    if(head==NULL){
        newNode->prev=NULL;
        head=newNode;
        return;
    }
    struct Node *temp=head;
    while(temp->next!=NULL)
        temp=temp->next;
    temp->next=newNode;
    newNode->prev=temp;
}
void display(){
    struct Node *temp=head;
    while(temp!=NULL){
        printf("%d <-> ",temp->data);
        temp=temp->next;
    }
    printf("NULL\n");
}
int main(){
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);
    display();
}
```

```
Output

10 <-> 20 <-> 30 <-> NULL
```

Lab 8: Circular Linked List

Insert At Beginning:

main.c

Share Run

```c
#include<stdio.h>
#include<stdlib.h>
struct Node{
    int data;
    struct Node *next;
};
struct Node *head=NULL;
void insertBeginning(int value){
    struct Node *newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=value;
    if(head==NULL){
        newNode->next=newNode;
        head=newNode;
        return;
    }
    struct Node *temp=head;
    while(temp->next!=head)
        temp=temp->next;
    newNode->next=head;
    temp->next=newNode;
    head=newNode;
}
void display(){
    struct Node *temp=head;
    do{
        printf("%d -> ",temp->data);
        temp=temp->next;
    }while(temp!=head);
    printf("(back to head)\n");
}
int main(){
    insertBeginning(10);
    insertBeginning(20);
    insertBeginning(30);
    display();
}
```

Output

```
30 -> 20 -> 10 -> (back to head)
```

LAB 9:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
struct Node* create(int x) {
    struct Node* new = malloc(sizeof(struct Node));
    new->data = x;
    new->left = new->right = NULL;
    return new;
}
struct Node* insert(struct Node* root, int x) {
    if (root == NULL) return create(x);
    if (x < root->data)
        root->left = insert(root->left, x);
    else
        root->right = insert(root->right, x);
    return root;
}
```

```c
void preorder(struct Node* root) {
    if (root) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
int main() {
    struct Node* root = NULL;
    root = insert(root, 10);
    insert(root, 5);
    insert(root, 15);
    printf("Preorder: ");
    preorder(root);
    return 0;
}
```

```
Preorder: 10 5 15
```

LAB 10:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
struct Node* create(int x) {
    struct Node* new = malloc(sizeof(struct Node));
    new->data = x;
    new->left = new->right = NULL;
    return new;
}
struct Node* find_lca(struct Node* root, int n1, int n2) {
    if (root == NULL) return NULL;
    if (root->data == n1 || root->data == n2)
        return root;
    struct Node* left = find_lca(root->left, n1, n2);
    struct Node* right = find_lca(root->right, n1, n2);
    if (left && right) return root;
    return left ? left : right;
}
```

```c
int main() {
    struct Node* root = create(10);
    root->left = create(5);
    root->right = create(15);
    struct Node* lca = find_lca(root, 5, 15);
    printf("LCA: %d", lca->data);
    return 0;
}
```

```
LCA: 10
```

LAB 11:

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};
struct Node* create(int x) {
    struct Node* new = malloc(sizeof(struct Node));
    new->data = x;
    new->left = new->right = NULL;
    return new;
}
void find_grandchildren(struct Node* root, int val) {
    if (root == NULL) return;
    if (root->data == val) {
        if (root->left) {
            if (root->left->left) printf("%d ", root->left->left->data);
            if (root->left->right) printf("%d ", root->left->right->data);
        }
        if (root->right) {
            if (root->right->left) printf("%d ", root->right->left->data);
            if (root->right->right) printf("%d ", root->right->right->data
                );
                );
        }
        return;
    }
    find_grandchildren(root->left, val);
    find_grandchildren(root->right, val);
}
int main() {
    struct Node* root = create(10);
    root->left = create(5);
    root->right = create(15);
    root->left->left = create(2);
    root->left->right = create(7);
    printf("Grandchildren: ");
    find_grandchildren(root, 10);
    return 0;
}
```

Output

```
Grandchildren: 2 7
```

Lab 12:

```c
#include <stdio.h>
#define MAX 10
int adj[MAX][MAX] = {0};
void add_edge(int v, int w) {
    adj[v][w] = 1;
    adj[w][v] = 1;
}
int main() {
    int n = 3;
    add_edge(0, 1);
    add_edge(1, 2);
    printf("Adjacency Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", adj[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
Output

Adjacency Matrix:
0 1 0
1 0 1
0 1 0
```

LAB 13:

```c
#include <stdio.h>
#define MAX 10
int adj[MAX][MAX] = {0};
int visited[MAX] = {0};
void add_edge(int v, int w) {
    adj[v][w] = 1;
    adj[w][v] = 1;
}
void dfs(int v) {
    printf("%d ", v);
    visited[v] = 1;
    for (int i = 0; i < MAX; i++) {
        if (adj[v][i] && !visited[i]) {
            dfs(i);
        }
    }
}
int main() {
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(0, 2);
    printf("DFS: ");
    dfs(0);
    return 0;
}
```

```
Output

DFS: 0 1 2
```

LAB 14:

main.c

```c
#include <stdio.h>
#define MAX 10
int adj[MAX][MAX] = {0};
int visited[MAX] = {0};
void add_edge(int v, int w) {
    adj[v][w] = 1;
    adj[w][v] = 1;
}
int dfs_cycle(int v, int parent) {
    visited[v] = 1;
    for (int i = 0; i < MAX; i++) {
        if (adj[v][i]) {
            if (!visited[i]) {
                if (dfs_cycle(i, v))
                    return 1;
            } else if (i != parent) {
                return 1;
            }
        }
    }
    return 0;
}
```

```c
int detect_cycle(int n) {
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            if (dfs_cycle(i, -1))
                return 1;
        }
    }
    return 0;
}
int main() {
    int n = 3;
    add_edge(0, 1);
    add_edge(1, 2);
    add_edge(2, 0);
    if (detect_cycle(n))
        printf("Cycle Detected\n");
    else
        printf("No Cycle\n");
    return 0;
}
```

```
Output
Cycle Detected
```

LAB 15:

```c
#include <stdio.h>
#define MAX 10
int network[MAX][MAX] = {0};
void add_friendship(int u1, int u2) {
    network[u1][u2] = 1;
    network[u2][u1] = 1;
}
int main() {
    int n = 3;
    add_friendship(0, 1);
    add_friendship(1, 2);
    printf("Friendship Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d ", network[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
Output
Friendship Matrix:
0 1 0
1 0 1
0 1 0
```